- Postup při návrhu databáze
- ER diagram a jeho části
- Klíče v relačních databázích
- Integrita databáze
- Organizace dat v databázi a indexace
- Normalizace relační databáze
- MySQL a nástroje pro správu databáze
- Jazyk SQL
- Tvorba databází a tabulek v MySQL
- Dotazy jazyka SQL
- Pokročilé možnosti MySQL

05

Tvorba a správa relační databáze

VÝUKOVÝ TEXT PRO STUDENTY INFORMAČNÍCH TECHNOLOGIÍ

Marek Lučný, SŠPU OPAVA 2015

A Postup při návrhu databáze

Promyšlené postupy a metody návrhu databáze byly vypracovány v rámci oboru nazývaného **softwarové inženýrství**. Jeho hlavním cílem je návrh, tvorba a údržba počítačových programů, tedy i databázových aplikací. Návrh databáze probíhá v několika fázích, kdy postupujeme od abstraktního směrem ke konkrétnímu.

FÁZE NÁVRHU DATABÁZE

1. Sběr požadavků a analýza. Cílem této přípravné fáze je shromáždit požadavky na systém (zejména na vstupní a výstupní informace), prostudovat a zanalyzovat potřebnou dokumentaci (normy, formuláře, zprávy, organizační pokyny), oslovit uživatele a zjistit jejich představy o budoucím systému.

2. Konceptuální model databáze. Výsledkem má být konceptuální schéma databáze, nezávislé na výběru DBMS, vyjádřené pomocí vysokoúrovňových datových modelů a zobrazené např. formou **ER** nebo **UML diagramů**.

3. Výběr databázového systému. Následuje volba vhodného typu DBMS, která by měla zohlednit charakter ukládaných dat i prováděných operací a samozřejmě vzít v potaz ekonomická i technická hlediska.

4. Logický model databáze. Během této fáze dochází k mapování konceptuálního návrhu do podoby konkrétního datového modelu podle zvoleného typu DBMS (vzniká např. relační, objektově orientovaný nebo hybridní datový model). V případě relačního datového modelu provádíme tzv. **normalizaci**.

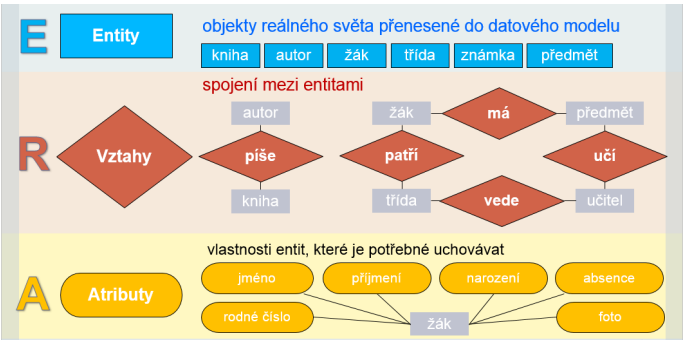
5. Fyzický návrh databáze. Dochází k převedení modelu do podoby konkrétní funkční databáze v některém z databázových systémů – v případě relačních databází např. MySQL, MS Access, Oracle apod.

6. Implementace databázového systému a ladění. Je prováděno testování připravené databáze i aplikace na vzorcích dat a odstraňují se nalezené chyby. Výsledkem by mělo být předání odladěného systému zákazníkovi.

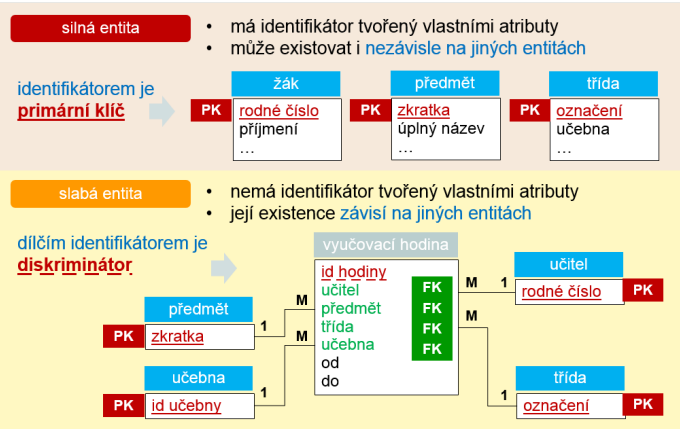
B ER diagram a jeho části

Pro návrh konceptuálního modelu databáze se nejčastěji využívá tzv. **ER model** (*Entity-Relationship*). Je založen na pojetí světa jako množiny **entit** (*entities*) a **vztahů** (*relationships*) mezi nimi. Popisuje data ve statické podobě a nenaznačuje, jaké operace budou s daty probíhat. Někdy je tento model označován i zkratkou **ERA** – jeho třetím základním prvkem totiž mohou být **atributy**. Grafické vyjádření ER modelu představuje **ER diagram** - diagram entit a vztahů.

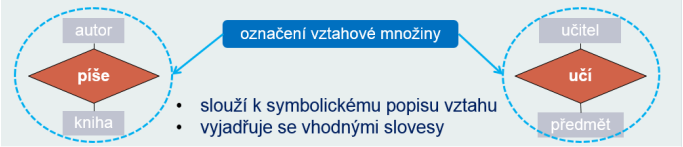
ZÁKLADNÍ PRVKY ER DIAGRAMU

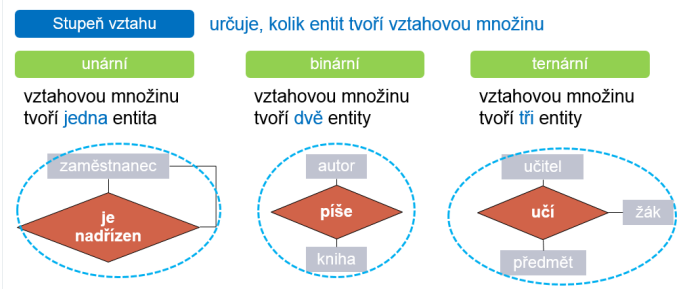


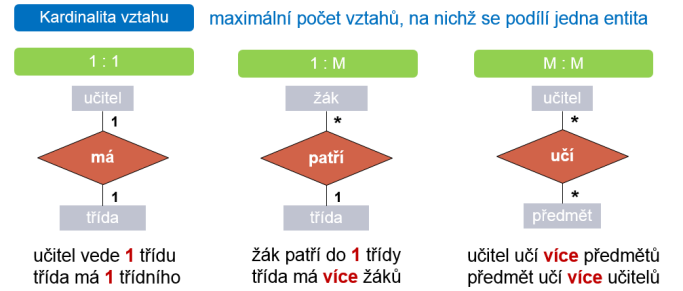
Entita. Objekt, který je schopný vlastní existence a dá se jednoznačně identifikovat. Může jít o objekt reálný (student, učebna, třída, kniha, autor), nebo abstraktní, který zachycuje určitou souvislost mezi entitami (vyučovací hodina, výpůjčka). Entity označujeme podstatnými jmény a zobrazujeme obdélníky.



Vztah zachycuje spojení mezi dvěma nebo více entitami. V diagramu se označují slovesy (např. autor píše knihu) a zobrazují se jako kosočtverce propojené čárami vedoucími ke každé entitě vztahu.

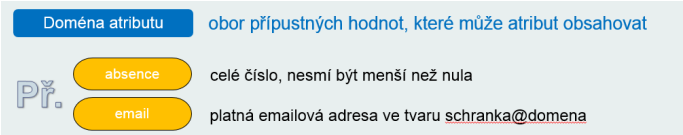






Atribut představuje určitou vlastnost entity (někdy i vztahu) - například entita student může obsahovat atributy jméno, příjmení, absence apod. Atributy se zobrazují jako elipsy propojené čarou s jejich vlastnící entitou. Každá entita (pokud nejde o slabou entitu) by měla mít jeden nebo více atributů, které ji jednoznačně identifikují a tvoří tzv primární klíč.

Doménou atributu rozumíme obor přípustných hodnot, které může daný atribut obsahovat - např. atribut narozen v entitě žák připouští pouze hodnoty v podobě platného data, které nesmí být z budoucnosti nebo vzdálené minulosti.



C Klíče v relačních databázích

Velmi důležitý prvek představují v relačních databázích **klíče**. S jejich pomocí můžeme nejen jednoznačně identifikovat jednotlivé záznamy, ale rovněž zajistit spojení mezi tabulkami.

Primární klíč (*primary key*) je pole (v případě složeného primárního klíče kombinace polí), které umožňuje jednoznačně identifikovat každý záznam v databázové tabulce. V každé tabulce by měl být definovaný právě jeden primární klíč, čímž je zajištěna tzv. **entitní integrita**. Základní vlastností orimárního klíče je, že obsahuje vždy nenulovou hodnotu, která je v rámci tabulky jedinečná. Databázový systém by měl být navržen a udržován tak, aby se primární klíč založeného záznamu nikdy nemusel měnit.

Typickým příklady přirozených primárních klíčů mohou být inventární číslo v případě evidence majetku, ISBN v tabulce knih, SPZ automobilů apod. Pokud není k dispozici žádný přirozený primární klíč, nebo je takový kandidátní klíč příliš složitý nebo problematicky použitelný (jako v případě rodného čísla, které je považováno za citlivý osobní údaj), nabízí se řešení s využitím automatického identifikačního čísla, které bývá obvykle označováno zkratkou ID.

Cizí klíč (*foreign key*) zajišťuje v relačních databází vztah mezi dvěma tabulkami. Jedná se o pole (zřídka kombinaci polí), jehož hodnoty odpovídají hodnotám obsaženým v primárním klíči některé z tabulek. Je tím vytvořeno spojení jednoho nebo více sloupců se sloupcem nebo více sloupci jiné („cizí“) tabulky, což je označováno jako **reference**.

Příkladem použití cizích klíčů může být již uvedená tabulka vyučovací hodina, která obsahuje hned čtyři cizí klíče odvozené z primárních klíčů „cizích“ (rodičovských) tabulek předmět, učebna, učitel a třída.

D Integrita databáze

Kvalita informací, které jsou získávány z databáze, závisí na přesnosti a celistvosti uložených dat – tzv. **integritě databáze**. Ta může být zajištěna na několika úrovních.

Doménová integrita. Souvisí s doménou atributu. Zajišťuje se volbou vhodného datového typu pro dané pole a validací vstupních dat podle předem daných pravidel. Když zadaný údaj neodpovídá pravidlům, nemůže být uložen a uživateli je to oznámeno chybovým hlášením. Ideálním doplněním validace je nastavení vstupní masky.

Entitová integrita. Každý záznam v databázi musí být jednoznačně identifikován primárním klíčem, který nesmí mít nulovou hodnotu.

Referenční integrita. Nesmí dojít k narušení logických vazeb mezi záznamy v různých tabulkách. Například změna nebo vymazání údaje v tabulce třída by mohla způsobit problémy v tabulce student u těch žáků, kteří patřili do dotyčné třídy. Tomu má zabránit mechanismus omezení cizích klíčů, pro který v SQL existuje klauzule CONSTRAINT:

```
[CONSTRAINT [název omezení]] FOREIGN KEY [klíč]
(sloupec tabulky nebo seznam sloupců oddělený čárkami)
REFERENCES jméno_tabulky (sloupec nebo seznam sloupců
oddělený čárkami)
[ON DELETE akce]
[ON UPDATE akce]
```

Cizí klíč umožňuje definovat akce, které mají nastat při pokusu o změnu nebo mazání záznamů v cizí tabulce:

CASCADE – pro klauzuli ON UPDATE se v dceřiné tabulce adekvátně změní odkazy na cizí klíč; v případě ON DELETE se záznamy odkazující na smazaný klíč smažou také.

Většina vyspělejších databázových systémů umožňuje tuto akci aplikovat zřetězeně.
SET DEFAULT – při změně nebo smazání rodičovského klíče se reference v dceřiné tabulce nastaví na hodnotu, kterou má definovanou jako výchozí.
SET NULL – jako SET DEFAULT, kde výchozí hodnotou je hodnota NULL.
RESTRICT – pokud v dceřiné tabulce existují záznamy odkazující na cizí klíč, pak odpovídající záznam v rodičovské tabulce nepůjde změnit nebo smazat.
NO ACTION – v konečném efektu stejné jako RESTRICT; rozdíly interpretace se liší podle jednotlivých databázových systémů: v některých akce NO ACTION na rozdíl od **RESTRICT** nevypisuje chybové hlášení.

Transakční integrita. V případě některých operací je k jejich úspěšnému završení nutné správně provést několik souvisejících kroků. Takové složitější operace jsou nazývány **transakce**. Pokud v průběhu transakce dojde k chybnému provedení určitého kroku (chybou uživatele nebo systému), je celá transakce stornována a všechny již provedené změny (například i zápisy do různých tabulek) vráceny do původního stavu. Příkladem využití v praxi je převod peněz mezi dvěma účty.

SYSTÉM ZÁMKŮ

Integrita databáze může být narušena rovněž současným přístupem více uživatelů k jednomu záznamu, zejména při současném pokusu o změnu údajů v některém sloupci, případně o vymazání jednoho či více záznamů. Z toho důvodu jsou v databázích používány tzv. zámky (lock), které dočasně uzamknou část databáze, s níž právě pracuje nějaký uživatel. Databáze často podporují různé typy uzamykání, např. na úrovni záznamů (row-level locking), kdy je uzamčen celý řádek tabulky, dokud je s ním pracováno, nebo na úrovni sloupců (column-level locking), kdy dojde k uzamčení právě měněného pole záznamu (sloupce).

E Organizace dat v databázi a indexace

Základní funkcí databázového systému je co nejefektivněji poskytovat data. Rychlé a přesné splnění požadavků uživatelů je závislé na uspořádání dat.

Ve starších databázích se používalo **sekvenční uspořádání**, v němž byly záznamy fyzicky seřazeny jeden za druhým podle své klíčové položky. Odpovídalo to tehdejšímu dávkovému zpracování úloh s využitím médií se sekvenčním přístupem (děrná páska, magnetická páska).

V případě **indexovaného uspořádání** je pro urychlení přístupu k uloženým záznamům využíván tzv. **index** - speciální uspořádaná datová struktura (velmi často B-strom), která obsahuje ukazatele na paměťová místa s daty. Index je obvykle spojen s konkrétním sloupcem tabulky a jeho hlavním účelem je zrychlit operace vyhledávání a řazení dat. Například v případě výběrového dotazu, který se týká indexovaných sloupců, není tabulka prohledávána podle toho, jak jsou za sebou řádky uloženy, ale pomocí informací uložených v paměťovém prostoru indexu je přistupováno přímo k požadovaným řádkům tabulky.
Každý index zpomaluje operace, které mění obsah indexovaných sloupců (například SQL příkazy UPDATE, INSERT), proto je třeba promyšleně indexovat jen ty sloupce tabulky, které jsou často prohledávány. Rozhodně nemá smysl indexovat dlouhé texty, binární data, číselníky o stálém počtu položek.

DRUHY INDEXŮ V MYSQL

PRIMARY. Tento primární index tvoří sloupec (nebo kombinace více sloupců), které obsahují primární klíč, v tabulce se může vyskytovat nejvýše jednou.
UNIQUE. Opět zajišťuje jedinečné hodnoty (PRIMARY je vlastně zvláštní případ typu UNIQUE), v tabulce však může být definováno více unikátních indexů.
INDEX (někdy SECONDARY). Je tvořen pro sloupce, které obsahují sekundární (vyhledávací) klíč. Definicí jednoho či více indexů tohoto typu v tabulce zajistíme optimalizaci vyhledávání pro zvolené sloupce (nazahrnujeme mezi ně ty sloupce, kterým již byly přiřazeny indexy typu PRIMARY nebo UNIQUE).
FULLTEXT. Využívá se u sloupců obsahující delší text k optimalizaci full-textové ho vyhledávání. Konkrétní řešení je závislé na databázovém systému.

PRÁCE S INDEXY V MYSQL

Indexovat jsou nutná pole, podle nichž se vyhledává nejčastěji, zejména primární klíče (v MySQL je indexován automaticky).

Jeden index může být vytvořen i z několika polí (např. Prijmeni a Jmeno).
- Vytvoření indexu - CREATE INDEX NazevIndexu ON Tabulka (seznam polí);
- Přidání indexu – ALTER TABLE Zakaznici ADD INDEX (IDXprijmeni);
- Odstranění indexu - DROP INDEX NazevIndexu ON Tabulka;

F Normalizace relační databáze

Relační datový model organizuje data do tzv. **relací** (tabulek).Relaci tvoří **záznamy** (řádky tabulky) s pevně stanovenými **atributy** (sloupce tabulky). Každý sloupec má definován jednoznačný název a doménu - obor přípustných hodnot. Záznam se stává n-ticí (řádkem) tabulky. Mezi jednotlivými tabulkami mohou být prostřednictvím primárních a cizích klíčů udržovány vazby. Kolekce více tabulek, jejich funkčních vztahů, indexů a dalších součástí tvoří relační databázi.

NORMALIZACE

V souvislosti s návrhem relační databáze se velmi často mluví o tzv. **normalizaci**. Jejím cílem je optimalizace databáze, spočívající hlavně v odstranění redundance - nadbytečného opakovaného ukládání stejných dat. Toho se dosahuje rozdělováním relací na více tabulek a jejich vzájemným propojením.

Normalizace se provádí podle pravidel, které nazýváme **normální formy**. Normální formy jsou označovány čísly, jež znamenají úroveň provedené normalizace (např. 1NF = *první normální forma*). Ve většině případů si vystačíme s prvními třemi normálními formami, pro speciální situace existují čtvrtá i pátá normální forma a tzv. *Boyce/Coddova forma*, která bývá pokládána za variaci třetí normální formy. V čím vyšší normální formě se databáze nachází, tím více by se měla blížit k optimálnímu relačnímu datovému modelu. Ani to však ještě nezaručuje, že výsledný databázový systém bude vysoce efektivní a funkční.

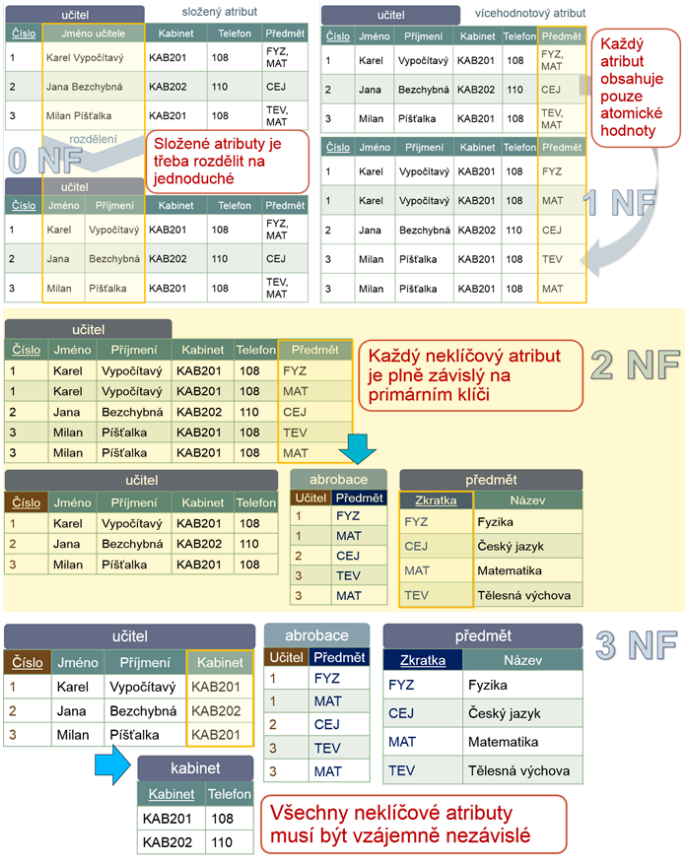
NORMÁLNÍ FORMY

0NF - Nenormalizovaná relace.

1NF - Každý atribut v relaci obsahuje pouze atomické hodnoty.

2NF - Každý neklíčový atribut je plně závislý na primárním klíči.

3NF - Všechny neklíčové atributy musí být vzájemně nezávislé.



G MySQL a nástroje pro správu databáze

MySQL je databázový systém, vytvořený švédskou firmou MySQL AB, nyní vyvíjený a vlastněný společností Oracle. Jde o multiplatformní databázový systém (lze jej instalovat na Linux, MS Windows i další operační systémy) šířený jako open source pod licencí GPL. Především díky tomu stále patří k nejvíce využívaným databázovým serverům na Internetu. Velmi oblíbená a často nasazovaná je kombinace Linux, MySQL, PHP a Apache jako základní software webového serveru (technologie LAMP).

MySQL byl od počátku optimalizován především na rychlost, a to i za cenu některých zjednodušení. Oproti dalším významným databázovým systémům má jen jednoduché způsoby zálohování a omezená je i podpora některých vyspělejších technologií. Funguje na principu klient/server - každé klientské připojení dostane uvnitř serverového procesu vlastní vlákno (thread). Autentizace je založena na uživatelském jménu, heslu a identifikaci hostitelského počítače. Nabízí několik typů databázových úložišť (storage engine), které se liší svými možnostmi, použitím a způsobem ukládání dat do souborů. Nejčastěji bývají využívány starší jednoduchý typ úložiště MyISAM a modernější typ InnoDB, který podporuje používání cizích klíčů, transakcí, uzamykání na úrovni řádků atd.

MySQL ukládá každou databázi do samostatných podadresářů označených názvem databáze. Každá vytvořená tabulka je uložena do souborů s příponami .FRM (obsahuje definice tabulky), .MYD (data tabulky) a .MYI (klíče tabulky). MySQL je závislá na diskovém systému; v případě instalace na Windows se velikost písmen v názvech tabulek a databází nerozlišuje, na unixových systémech se velikost písmen rozlišuje. Databáze i tabulky mohou být pojmenovány téměř libovolně a obsahovat všechny běžné znaky Unicode. V MySQL se pak takové jméno uvádí ve zpětných jednoduchých uvozovkách (`).

Konfigurace MySQL databáze je uložená v textovém souboru my.ini. V případě, že máme patřičná oprávnění na serveru, můžeme se prostřednictvím změny parametrů konfiguračního souboru pokusit optimalizovat nastavení systému. Nemalé navýšení výkonu databáze je možné dosáhnout aktivací cache (vyrovnávací paměti) jednak pro opakované výběrové dotazy, jednak pro indexování tabulek. Další možnosti zvýšení výkonu nabízí nastavení jednotlivých vláken nebo paralelně otevřených spojení. Vše můžeme konfigurovat změnou hodnot parametrů jednotlivých proměnných v souboru my.ini. Efektivitu nastavení můžeme sledovat pomocí informačních SQL dotazů typu SHOW STATUS.

NÁSTROJE PRO SPRÁVU MYSQL

Konzole MySQL. Součástí distribuce MySQL je řádkový klient mysql(.exe) a řádkový administrátor mysqladmin(.exe). Oba soubory se nacházejí v podadresáři /bin instalace SQL serveru. Spustit konzoli MySQL můžeme příkazem:

mysql.exe -h počítač -u uživatel -p heslo databáze

např. **mysql -u root -p** (spustí konzoli s právy standardního správce)

V řádkovém klientu je možné zadávat SQL dotazy zakončené středníkem.

Řádkový administrátor, který slouží ke správě MySQL serveru z příkazového řádku, můžeme spustit příkazem:

mysqladmin.exe -h počítač -u uživatel -p heslo seznam_příkazů

např. **mysqladmin -u root -p variables** (vypíše seznam proměnných serveru)

PHPMyAdmin. Populární aplikace napsaná v jazyce PHP umožňuje jednoduchou správu databází MySQL prostřednictvím webového rozhraní. Pomocí ní můžeme provádět všechny běžné operace s databázemi a tabulkami, včetně jejich vytváření, odstraňování, změn struktury, manipulací s daty atd. Efektivně lze rovněž spravovat přístupová práva uživatelů, provádět import i export a zadávat vlastní SQL dotazy. Obdobnými alternativními nástroji jsou např. Adminer (jde opět o webovou aplikaci) nebo MySQL administrator (okenní aplikace).

MySQL Workbench. Multiplatformní open source nástroj od autorů MySQL, který můžeme použít především k vizuálnímu návrhu logického modelu databáze, ale integruje v sobě i administrační funkce.

MySQL Workbench je možné propojit s jednotlivými MySQL servery a využít buď funkci **Forward Engineering** (dopředného inženýrství), kdy z kompletního návrhu ER diagramu necháme automaticky vytvořit funkční MySQL databázi, nebo naopak pomocí funkce **Reverse Engineering** (zpětné inženýrství) dosáhneme vygenerování ER diagramu z již hotové databáze.

Program rovněž zahrnuje *Schema Synchronization and Comparison utilities* - speciální nástroje, které mají návrhářům ulehčit složitý proces udržování různých verzí databází a provádění jejich změn (*change management*).

Další zajímavou funkcí aplikace je automatické *vytváření dokumentace* k databázím v podobě HTML i prostého textového souboru.

H Jazyk SQL

SQL (*Structured Query Language = jazyk strukturovaných dotazů*) je databázový dotazovací jazyk, který se využívá ve většině relačních databází. Vyvíjel se od 70. let pod záštitou firmy IBM (databázový jazyk SEQUEL), od 80. let začal na standardu databázového jazyka pracovat americký standardizační institut ANSI (American National Standards Institute). První verze tohoto standardu, SQL-86, byla přijata i mezinárodní standardizační organizací ISO (International Standards Organization) roku 1986. V 90. letech vzniklo konsorcium výrobců databází SQL Access Group, které publikovalo normalizovaný komerční standard SQL - variantu jazyka, které bude rozumět (nebo se jí řídit) každý důležitý relační databázový produkt. Standardy podporuje prakticky každá relační databáze, ale obvykle nejsou implementovány vždy všechny požadavky normy. A naopak, každá z nich obsahuje prvky a konstrukce, které nejsou ve standardech obsaženy. Přenositelnost SQL dotazů mezi jednotlivými databázemi je proto omezená.

VLASTNOSTI SQL

Neprocedurálnost (deklarativnost). Pomocí SQL popíšeme, jaká data chceme v databázi najít, ale nikoli, jak to má počítač provést (na rozdíl od procedurálních programovacích jazyků – C, Pascal, Java).

Relační úplnost. Prostřednictvím SQL můžeme provést všechny databázové operace (od založení databáze, přes vytváření struktury tabulek, stanovení integritních omezení, vkládání a zobrazení dat, až po definování práv uživatelů).

Neobsahuje prostředky pro formátování výstupů, pouze pro definici jejich obsahu - obrazovkové formuláře, tiskové sestavy, uživatelská menu apod. musí zajistit uživatelská aplikace (např. v PHP).

ROZDĚLENÍ JAZYKA SQL

Struktura jazyka SQL odpovídá obecnému členění databázových jazyků:

DDL (*Data Definition Language - jazyk pro definici dat*). Příkazy slouží k vytváření struktury databáze – tabulek, indexů, pohledů a další objektů. Vytvořené struktury lze také upravovat, doplňovat a mazat.

CREATE	vytváření nových objektů (CREATE DATABASE, CREATE TABLE, CREATE VIEW)
ALTER	změny existujících objektů (ALTER TABLE)
DROP	odstraňování objektů (DROP DATABASE, DROP TABLE)
RENAME	přejmenování objektů

DML (*Data Manipulation Language - jazyk pro manipulaci s daty*). Příkazy pro získávání a úpravy dat uložených v databázi.

INSERT	vkládání nových dat do tabulky
REPLACE	nahrazení existujících záznamů v tabulce
UPDATE	změna dat v tabulce
DELETE	odstraňování záznamů z tabulky
TRUNCATE	odstraní všechny záznamy, ponechá strukturu tabulky

DCL (*Data Control Language - jazyk pro správu dat*). Do této skupiny patří příkazy pro nastavování přístupových práv a řízení transakcí.

GRANT	příkaz pro přidělení oprávnění uživateli k určitým objektům
REVOKE	příkaz pro odnětí práv uživateli

Speciální skupinu tvoří příkazy pro řízení tzv. transakcí. Transakci může tvořit několik operací, které musejí proběhnout všechny, aby mohla být celá transakce platná. V případě jakékoliv chyby je nutné transakci zrušit:

BEGIN	zahájení transakce
COMMIT	potvrzení transakce
ROLLBACK	zrušení transakce, návrat do původního stavu

DQL (*Data Query Language - dotazovací jazyk*). V jazyce SQL jde o nejčastěji používaný příkaz **SELECT**, který slouží k výběru dat.

I Tvorba databází a tabulek v MySQL

Vytvořit novou databázi a tabulku v MySQL můžeme buď pomocí grafických prostředků (např. v aplikacích PHPMyAdmin nebo MySQL Workbench), nebo můžeme použít SQL dotazů:

Příklad vytvoření databáze *skola*:

```
CREATE DATABASE skola;
```

Příklad vytvoření tabulky *zaci*, která bude obsahovat tato pole:
- id (primární klíč jako automaticky se zvyšující celé číslo, nenulové)
- jmeno (znakový řetězec o maximální délce 30 znaků, povinný - nenulový)
- narozeni (datum)
- prumer (desetinné číslo s pevnou řadovou čárkou, s 2 desetinnými místy)
- absence (nezáporné celé číslo v rozsahu 0 až 999, výchozí nastavení 0)

```
CREATE TABLE zaci (
  id INT PRIMARY KEY AUTO_INCREMENT NOT NULL,
  jmeno VARCHAR(30) NOT NULL,
  narozeni DATE,
  prumer DECIMAL(3,2),
  absence INT(3) UNSIGNED DEFAULT 0);
```

Příklady změny struktury tabulky:

```
ALTER TABLE zaci ADD poznamka TEXT AFTER narozeni;
ALTER TABLE zaci CHANGE poznamka fotka BLOB;
ALTER TABLE zaci DROP fotka;
ALTER TABLE zaci ADD INDEX jmeno(jmeno);
```

Příklad odstranění databáze *skola*:

```
DROP DATABASE skola;
```

Příklad odstranění tabulky *zaci*:

```
DROP TABLE zaci;
```

VYBRANÉ DATOVÉ TYPY V MYSQL

INT – celočíselný typ, v závorce lze stanovit délku (počet míst); příznak unsigned (bez znaménka) znamená, že je možné vkládat pouze nezáporné celočíselné hodnoty; příznak zerofill doplní neobsazená místa nulami; příznak auto_increment zajistí automatické zvyšování čísla v případě vložení nového záznamu (používá se pro automaticky generované hodnoty primárních klíčů).

DECIMAL – desetinné číslo s pevnou řadovou čárkou; v závorce je nutné specifikovat celkový počet míst a počet míst za desetinnou čárkou/tečkou; př. DECIMAL(5,2); hodí se zejména pro ukládání finančních hodnot.

FLOAT – desetinné číslo s plovoucí řadovou čárkou; v tomto případě je nutné kalkulovat s možnou nepřesností způsobenou zaokrouhlováním (souvisí s principem ukládání reálných čísel v počítačové paměti).

VARCHAR – znakový řetězec s proměnnou (variabilní) délkou; maximální délku řetězce určuje číslo v závorce; př. VARCHAR(30).

CHAR – znakový řetězec s pevně stanovenou délkou; př. CHAR(11) – rodné číslo s lomítkem.

TEXT – pro uložení delšího souvislého textu bez předem specifikované délky.

DATETIME – uchovává úplnou informaci o datu a čase příslušné události s přesností na sekundu; rozsah 1000-01-01 00:00:00 až 9999-12-31 23:59:59.

DATE – lze analogicky uchovávat pouze datum.

TIME – uchovává čas události nezávisle na kalendářním datu. Platné časové hodnoty mimo povolený interval budou zkráceny na mezní hodnotu, neplatné budou nastaveny na nulovou hodnotu.

YEAR – ukládání informací o kalendářním datu v intervalu od 1901 do 2155.

TIMESTAMP – časové razítko, uchovává pouze hodnoty v intervalu od roku 1970 do 2037. Pole typu TIMESTAMP jsou inicializována aktuálním časem pouze v případě, že do nich není nic zapsáno (tj . v přidávacích a aktualizačních dotazech toto pole nepoužívat nebo inicializovat prázdnou hodnotou).

ENUM – výčtový typ, může obsahovat pouze jednu z deklarovaných hodnot; př. ENUM('ČT','Nova','Prima'); vhodný pro formulářový typ radio.

SET – množina, může obsahovat až 64 položek z deklarovaných hodnot; deklaruje se obdobně jako typ ENUM; vhodný pro formulářový prvek checkbox.

BLOB – datový typ pro ukládání binárních dat (obrázky, zvuky, videa).

J Dotazy jazyka SQL

VÝBĚROVÉ DOTAZY

Výběrové dotazy, které začínají klíčovým slovem SELECT, jsou nejpoužívanějším typem dotazů. Slouží k získávání informací z databází. Výsledkem dotazu (query), který může být proveden nad jednou nebo více tabulkami, je blok dat zvaný obvykle dynamická sada (result set). Při zadávání dotazu je nutné dodržovat správné pořadí klíčových slov tak, jak je dáno syntaxí:

Syntax dotazu SELECT:

```
SELECT
[ALL | DISTINCT | DISTINCTROW ]
atribut [, atribut ...]
[FROM tabulky]
[WHERE podmínky]
[GROUP BY {atribut | výraz} [ASC | DESC]]
[HAVING podmínky_pro_seskupení]
[ORDER BY {sloupec | výraz} [ASC | DESC], ...]
[LIMIT {od_záznamu, kolik_záznamů}]
```

Praktické příklady použití výběrových dotazů

Seznam čtenářů pocházejících z Opavy (podmínka **WHERE**), setříděný (**ORDER BY**) podle příjmení (vzestupně) a města (sestupně), omezený na prvních 5 záznamů (**LIMIT**):

```
SELECT prijmeni, jmeno, ulice, mesto, psc
FROM ctenar
WHERE mesto = 'Opava'
ORDER BY prijmeni ASC, mesto DESC
LIMIT 0,10;
```

Příklad propojení tabulek (**JOIN**) s využitím primárního a cizího klíče.

Výpis exemplářů knih zakoupených po 1. 1. 2000. Slovo AS představuje tzv. alias, dočasné pojmenování sloupce nebo tabulky pro účely dotazu.

```
SELECT K.nazev, E.koupeno AS `Datum nákupu`, E.cena
FROM exemplar AS E JOIN kniha AS K ON E.isbn = K.isbn
WHERE E.koupeno > '20000101';
```

Příklad seskupovacího dotazu (**GROUP BY**), použití tzv. agregační funkce (**COUNT**) a podmínky aplikované na seskupená data (**HAVING**).

Seznam čtenářů, kteří mají půjčenu více než 1 knihu:

```
SELECT cislo, COUNT(*)
FROM vypujcka
GROUP BY cislo
HAVING COUNT(cislo)>1;
```

Příklad použití **funkcí** a **poddotazu**.

Zobrazí jména těch čtenářů (ve tvaru první znak křestního jména a příjmení), kteří si dosud nevypůjčili žádnou knihu.

```
SELECT CONCAT(LEFT(jmeno,1),'. ',prijmeni)
FROM ctenar
WHERE cislo NOT IN (SELECT cislo FROM vypujcka)
```

Příklad použití klíčového slova **DISTINCT** a operátoru **LIKE**.

Vypíše seznam autorů začínajích písmenem B (autor se v seznamu nebude opakovat - DISTINCT), exempláře jejichž knih byly zakoupeny před rokem 2000.

```
SELECT DISTINCT K.autor
FROM exemplar AS E JOIN kniha AS K ON E.isbn = K.isbn
WHERE koupeno < '20000101' AND K.autor LIKE 'B%';
```

AKČNÍ DOTAZY

Provádějí změny dat v databázi. Patří mezi ně často používané dotazy INSERT (vložení dat), UPDATE (aktualizace dat) a DELETE (odstranění dat).

Praktické příklady použití dotazu INSERT

Nejdelší varianta dotazu, který vloží nový záznam do tabulky zaci. V tomto případě jsou nejprve vyjmenovány názvy sloupců a poté zadány příslušné hodnoty:

```
INSERT INTO zaci (id, jmeno, narozeni, vyska, vaha,
pohlavi) VALUES('','Jana','19981206',1.68,62,'F');
```

Nejkratší varianta dotazu, v níž pořadí hodnot musí odpovídat pořadí sloupců v tabulce:

```
INSERT INTO zaci VALUES('', 'Jana', '19981206',
1.68, 62, 'F');
```

Varianta s klíčovým slovem **SET**, která je obdobou dotazu UPDATE:

```
INSERT INTO zaci SET jmeno = 'Jana', narozeni =
'19981206', vyska = 1.68, vaha = 62, pohlavi = 'F';
```

Praktický příklad použití dotazu UPDATE

Nastaví hodnotu sloupce pohlaví na *F* pro všechny žáky, jejichž jméno je zakončeno písmenem *a* (**LIKE '%a'**).

```
UPDATE zaci SET pohlavi = 'F' WHERE jmeno LIKE '%a';
```

Praktický příklad použití dotazu DELETE

Vymaže záznamy žáků, u nichž není uvedeno datum narození (**IS NULL**).

```
DELETE FROM zaci WHERE narozeni IS NULL;
```

K Pokročilé možnosti MySQL

ULOŽENÉ PROCEDURY

Uložené procedury jsou sady příkazů SQL, které jsou uloženy na serveru a zkompilovány do spustitelné, binární formy. Server si pro uloženou proceduru vytvoří tzv. *prováděcí plán*, díky němuž je dosahováno rychlejšího spouštění dotazů, zejména při jejich opakovaném volání. Uložené procedury tak lze využít jak pro zrychlení databázových aplikací, tak i pro zvýšení jejich bezpečnosti (možnost lepší ochrany proti SQL injection, logování přístupů k datům atd.).

Uložené procedury jsou výbavou vyspělejších databázových systémů, v MySQL jsou podporovány až od verze 5. Každá uložená procedura je v MySQL spojena s konkrétní databází – s odstraněním databáze jsou odstraněny i uložené procedury. Vytvářejí se příkazem **CREATE PROCEDURE** a odstraňují příkazem DROP PROCEDURE.

```
mysql>DELIMITER //
CREATE PROCEDURE vlozit(id int, nazev varchar(50))
BEGIN
  INSERT INTO software values (id, nazev);
END //
mysql>DELIMITER ;
```

Tělo procedury obsahuje středník; odřádkování by tedy v příkazovém režimu způsobilo odeslání neúplné definice a procedura by nebyla vytvořena.V řádkovém klientu proto použijeme příkaz DELIMITER a změníme sekvenci znaků ukončujících příkaz za něco jiného než je středník: DELIMITER //

Vytvořenou proceduru lze volat pomocí příkazu CALL().

```
mysql>CALL vlozit(10, 'GIMP');
```

TRIGGERY

Trigger (spoušť) je uložená procedura, která se spouští v souvislosti s provedením nějakého akčního dotazu na tabulce. Může být spuštěna buď před provedením úpravy dat, nebo po zapsání změn dat do databáze. Každý trigger souvisí s jednou tabulkou - s odstraněním tabulky jsou odstraněny i triggery.

Trigger se tvoří pomocí příkazu **CREATE TRIGGER**. Jméno triggeru musí být pro danou databázi jedinečné (nelze mít dva shodně pojmenované triggery ani na dvou různých tabulkách). U každého triggeru musí být uvedeno, před nebo po jaké akci je spuštěn. Možnosti: BEFORE INSERT, BEFORE UPDATE, BEFORE DELETE, AFTER INSERT, AFTER UPDATE, AFTER DELETE.

Tělo spouště v MySQL začíná klauzulí **FOR EACH ROW**, že následující akce bude provedena tolikrát, kolik je řádků ovlivněných akčním dotazem. Triggery mají přístup k měněným datům prostřednictvím virtuálních tabulek old a new. Tabulka **old** obsahuje původní data (před jejich změnou) a tabulka **new** obsahuje konečná data (již změněná dotazy INSERT nebo UPDATE).

```
mysql>DELIMITER //
CREATE TRIGGER trMaxPlat
BEFORE UPDATE ON bezpecna
FOR EACH ROW
BEGIN
  IF new.plat > (old.plat * 1.1) THEN
    SET new.plat=old.plat;
  END IF;
END; //
mysql>DELIMITER ;
```